Brewing SaaS - Architecture & Implementation Plan (AI-first)

Status: v0.2 (living document)
Primary goal: ship a BrewersFriend-style product with **native mobile apps** (marketing + brew-day reliability) and a **desktop-first web app** (workhorse), built with an **AI-first workflow** (Cursor + GPT-5.2-codex).
Guiding principle: keep the backend *boring* (monolith, predictable patterns), make offline reliable without early sync complexity, and keep decisions easy to review and evolve.

---

1. Product goals and non-goals

Goals

- **Native apps are mandatory** (iOS/Android app stores). Native is not a “PWA replacement”; it is the differentiator:
  - brew-day must work with poor/no connectivity
  - offline-first logging
  - store presence for marketing
- **Desktop web is the workhorse** for serious workflows (recipe building, analysis, administration).
- **AI-first development**:
  - reduce cognitive load and context switching
  - prefer mainstream conventions and type safety
  - code is shipped with tests (unit + e2e) from day 1
- **Keep billing simple**:
  - start with Stripe (few tiers)
  - keep Paddle as a later option (billing adapter boundary)

Non-goals (for v0)

- Level 3 local-first sync (multi-device conflict-free merges) unless demanded by customers.
- Complex GraphQL schema governance (start with boring REST + projection).
- Microservices, event sourcing, CQRS, custom infrastructure.

---

2. Big picture architecture

Chosen stack

- **Web app (desktop + mobile web):** Next.js + React + TypeScript
- **Native apps (iOS/Android):** React Native + Expo + TypeScript
- **Backend monolith API:** Node.js + Fastify + TypeScript
- **ORM / schema management:** Prisma
- **Central database:** PostgreSQL
- **Offline database (device):** SQLite (native apps)
- **Reverse proxy (“doorman”):** Nginx
- **Local dev:** Docker Compose (routing parity: Nginx container included)
- **CI/CD:** GitHub + GitHub Actions
- **Deployment:** Bare metal VPS initially; Postgres on same VPS initially; dedicated DB VPS later if needed

Why these choices (summary)

- **TypeScript everywhere** reduces context switching and increases AI effectiveness (types catch mistakes).
- **Fastify** keeps backend simple and fast; encourages explicit composition.
- **Prisma** provides schema-driven workflow and strong developer ergonomics.
- **PostgreSQL** is a strong general-purpose DB, great for evolving SaaS needs.
- **SQLite on device** provides true offline persistence (brew-day reliability).
- **Nginx** is familiar and great for TLS, routing, and deployment predictability.
- **Option 1: separate web and mobile UIs** preserves best-in-class UX on each platform while still

sharing core logic and contracts.

---

3. Request flow and responsibilities (LEMP analogy)

Nginx is the doorman. Next.js is the web app. Fastify is the backend.

Runtime responsibilities

- **Nginx**
  - TLS termination (or delegated to a managed edge later)
  - route `/api/*` to Fastify
  - route `/` to Next.js
  - rate limits, request size caps, basic hardening
- **Next.js (web)**
  - web routing, SSR/CSR rendering, web UI
  - calls backend via `/api`
- **Fastify (API)**
  - authentication/authorization (ACL)
  - business rules
  - persistence (Postgres via Prisma)
  - Stripe webhooks + entitlement updates
  - background jobs (later)
- **Expo (mobile)**
  - offline-first UI
  - local SQLite persistence
  - sync queue pushing events to backend when online

---

4. Tenancy model and ACL (Magento-like)

Tenancy

- **Account/team-owned** model from day 1.
- Users belong to one or more accounts (clubs/breweries) with roles.

Roles

User-facing role names:

- `owner`
- `brewery-admin`
- `member`
- `viewer`

Implementation note:

- for database identifiers, use `brewery_admin` (underscore), while the UI can show `brewery-admin` (dash).

Authorization approach

- **Application-level authorization** (Fastify middleware + service-layer checks) is the baseline.
- **Row Level Security (RLS)** may be added later for defense-in-depth, but is not required for v0.

Data ownership rule

All domain tables include `account_id` . Every read/write is scoped by:

- authenticated `user_id`
- chosen `active_account_id`
- membership + role checks

Support tooling requirement: “login as brewery” (impersonation)

We want a support-only capability to reproduce user-reported bugs:

- “Login as brewery/account” (and/or “login as user within account”)
- must be **audited** (who impersonated whom, when, why)
- restricted to specific privileged support roles / feature flags
  Implementation is deferred; this is a design constraint to keep in mind.

---

5. Offline strategy: reliable Level 2 now, defer Level 3

Offline requirements

- Brew-day workflows must **never lose data**.
- Measurements and notes are stored locally immediately.

Level 2 offline (chosen)

- **Write locally first** (SQLite) with `sync_state = pending`.
- Sync loop retries until server acknowledges events.
- Server endpoints are **idempotent** by client-generated `event_id` (UUID).

Append-only events: the key simplification

Instead of “editing the same record”, brew-day data is modeled as immutable events:

- gravity/temp/pH measurements
- notes
- timer start/stop
- corrections (as new events referencing the superseded event)

This makes sync “add missing events” and avoids most conflicts.

Level 3 (deferred)

Multi-device offline edits with conflict resolution is not planned unless demanded.
Design still keeps the door open via:

- append-only event model
- versioning on mutable entities (recipes)
- explicit conflict policies if introduced later

---

6. API design: boring REST + projection

Base rule

Start with straightforward REST endpoints:

- `/accounts`, `/members`, `/recipes`, `/batches`, `/events`

Projection (GraphQL-like benefits without GraphQL)

- `?include=` for related data
- `?fields=` for selective field projection

Sync endpoints

- `POST /sync/push` - send local events batch (idempotent)
- `GET /sync/pull?since=` - fetch server-side changes since cursor/token

Public recipe submission (web + API)

We want a public-facing path to submit recipes (for community/public pages):

- Web form should be protected (captcha and rate limiting).
- The same capability must be possible via API (for automation/AI agents), protected by API keys/tokens and rate limits.
  Implementation is deferred; this is a design constraint.

---

7. Data model: overview and domain constraints

Core entities (high level)

- Account, AccountMember (roles)
- User, Device
- Recipe
- Batch
- BrewEvent (append-only)

Water profiles are part of the recipe (day-1 rule)

A water profile is not “external metadata”; it is a core part of the recipe and must obey:

- exactly **one water profile per recipe** (1:1)
- it cannot be linked to multiple recipes
- it can be **duplicated** to another recipe (copy creates a new profile record)
- water profile editing is limited to water correction:
  - only water amounts and salts/acids adjustments
  - do not edit grist/malt bill or hop schedule from water profile screens
- we maintain **a single source of truth** for values:
  - the water profile is the authoritative data
  - recipe views can display values copied/derived from the profile, but edits must happen in one place only

We will need database entities for:

- salts
- acids
- water profiles and water additions
  (chemistry is complex but manageable: we follow the John Palmer’s Water App "rules": they have proven to be super effective; the key constraint is “single source of truth” to keep sync and recalculations sane). pH, residual alkalinity, CaCo3 and all the chemistry "stack" must be taken into account. bob's app also an interesting insight.

Single source of truth principle (global rule)

We must avoid storing the same conceptual data in two editable places.
Reason: offline sync and recalculation become unreliable when updates do not propagate deterministically.

---

8. Brew-day steps, reminders, and safety

We want support for custom, user-defined brew-day steps (including safety steps):

- examples: “close LPG valve”, “sanitise pump”, “open chiller bypass”, etc.
- steps can be defined at:
  - brewery profile level (defaults)
  - specific batch/brew session level (overrides/additions)
- steps can trigger notifications/reminders (future)

This is a key product differentiator and a strong reason to prefer native apps.

---

9. Testing strategy and testability conventions

Principles

- Keep tests “boring” and useful.
- Put correctness into **core domain library** (pure functions).

Stack

- **Unit tests:** Vitest (packages/core, services/api)
- **Integration tests:** API + DB (compose-based test DB or testcontainers later)
- **E2E tests:** Playwright (web)
- **Mobile:** start with unit + a small number of integration tests; add heavier mobile e2e after flows stabilize

E2E selector convention: `data-testid` (balanced rule)

To keep Playwright reliable and reduce brittle selectors:

- add `data-testid` to workflow-critical elements:
  - primary buttons (save/create/delete/sync)
  - key form fields
  - key navigation items
  - modals/toasts that confirm important actions
- do not add `data-testid` to purely presentational components

Prefer accessibility selectors (`getByRole` , `getByLabel`) when stable, and use `data-testid` as the stable fallback for custom/duplicated/dynamic UI.

---

10. Deployment approach (VPS + Docker)

Local dev

- Docker Compose with:
  - `nginx`
  - `web` (Next.js dev)
  - `api` (Fastify dev)
  - `postgres`
- Expo mobile dev runs on host or a container depending on preference.

Production

- Bare metal VPS:
  - Nginx on host or container
  - Docker Compose running web/api
  - Postgres initially on same VPS
- Later:
  - move Postgres to dedicated VPS or managed provider

    - add PgBouncer when connection pressure appears
    - add read replicas when read-heavy scaling requires it

---

11. Cursor-ready implementation plan (phases)

Phase 0 - Repository and standards

- Create monorepo structure (`apps/` , `services/` , `packages/` , `infra/` , `docs/`)
- Add `docs/architecture.md` as source of truth for Cursor
- Shared TS configs + linting
- CI skeleton (typecheck + unit)

Phase 1 - Backend foundation (Fastify + Prisma)

- Bootstrap Fastify server with:
    - request context (`userId` , `activeAccountId` , role)
    - auth middleware
    - error handling + structured logs
- Prisma schema + migrations
- Seed scripts (dev)

Phase 2 - Accounts + ACL

- CRUD:
    - create account
    - invite/join members
    - role management (owner/brewery-admin/member/viewer)
- Enforce ACL centrally in service layer.

Phase 3 - Domain core + first features

- `packages/core` :
    - brewing calculations (pure functions)

    - validation and unit conversions
    - strong unit tests
- API endpoints for recipes and batches (scoped to account).

Phase 4 - Append-only events + sync

- Data model for `BrewEvent`
- `/sync/push`:
    - validate membership
    - idempotent insert by `event_id`
    - return ack list
- `/sync/pull`:
    - return changes since cursor (timestamp or server token)

Phase 5 - Web app (Next.js)

- Auth + account switcher
- Desktop-first UI for recipes/batches
- Playwright e2e baseline
- Mobile web responsive support + “Open in app” prompts

Phase 6 - Mobile app (Expo)

- Local SQLite schema mirrors event model
- Brew-day mode:
    - local event capture
    - sync queue + retries
- Deep links:
    - open specific batch from web links

Phase 7 - Billing (Stripe) + entitlements

- Stripe checkout flow
- Webhook ingestion (store StripeEvent idempotently)

- Entitlement updates
- Feature gating and plan limits (API enforced; UI mirrored)
- Keep provider adapter boundary so Paddle can replace later.

Phase 8 - Deployment hardening

- Nginx config (routing parity)
- Docker production compose
- secrets management
- monitoring and backups
- optional PgBouncer when scaling app instances

---

12. Living constraints (the “always apply” rules)

- Keep backend monolithic and boring.
- REST-first. Add include/fields before considering GraphQL.
- Offline reliability is a product guarantee (native apps).
- Append-only events for brew-day logs.
- Account/team tenancy + role-based ACL enforced server-side.
- Support-only “login as brewery/account” capability (audited).
- Public recipe submission (web + API), protected by captcha/keys and rate limits.
- Water profile is part of recipe (1:1) with single source of truth; salts/acids DB.
- Avoid duplicated editable data in multiple places (single source of truth).
- Brew-day custom steps + safety reminders are first-class (future notifications).
- Tests ship with features (unit + Playwright baseline).
- Add `data-testid` to workflow-critical UI elements (balanced rule).

Appendix A - Initial Prisma schema (v0.2)

```
enum AccountRole {
  owner
  brewery_admin
  member
  viewer
}

enum SubscriptionStatus {
  trialing
  active
  past_due
  canceled
  unpaid
  incomplete
}

enum BrewEventType {
  gravity
  temperature
  ph
  note
  timer_started
  timer_stopped
  timer_lap
}

model User {
  id            String         @id @default(uuid())
  email         String         @unique
  displayName   String?
  createdAt     DateTime       @default(now())
  updatedAt     DateTime       @updatedAt

  memberships   AccountMember[]
  devices       Device[]
  subscriptions Subscription[]
```

```
  entitlement   Entitlement?
}

model Account {
  id        String          @id @default(uuid())
  name      String
  createdAt DateTime        @default(now())
  updatedAt DateTime        @updatedAt

  members   AccountMember[]
  recipes   Recipe[]
  batches   Batch[]
}

model AccountMember {
  id        String      @id @default(uuid())
  accountId String
  userId    String
  role      AccountRole @default(member)
  createdAt DateTime    @default(now())

  account   Account     @relation(fields: [accountId], references: [id],
onDelete: Cascade)
  user      User        @relation(fields: [userId], references: [id],
onDelete: Cascade)

  @@unique([accountId, userId])
  @@index([userId])
}

model Recipe {
  id        String   @id @default(uuid())
  accountId String
  name      String
  style     String?
  version   Int      @default(1)
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
```

```
  account   Account  @relation(fields: [accountId], references: [id],
onDelete: Cascade)
  batches   Batch[]

  @@index([accountId])
}

model Batch {
  id          String   @id @default(uuid())
  accountId   String
  recipeId    String?
  name        String
  status      String   @default("planned")
  startedAt   DateTime?
  completedAt DateTime?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  account     Account  @relation(fields: [accountId], references: [id],
onDelete: Cascade)
  recipe      Recipe?  @relation(fields: [recipeId], references: [id],
onDelete: SetNull)
  events      BrewEvent[]

  @@index([accountId])
}

model Device {
  id         String   @id @default(uuid())
  userId     String
  deviceName String?
  platform   String?  // ios|android|web
  createdAt  DateTime @default(now())
  lastSeenAt DateTime?

  user       User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
```

```
  @@index([userId])
}

model BrewEvent {
  id          String        @id
  accountId   String
  batchId     String
  deviceId    String?
  createdById String?
  type        BrewEventType
  occurredAt  DateTime
  payload     Json
  createdAt   DateTime      @default(now())
  supersedesEventId String?

  batch       Batch         @relation(fields: [batchId], references: [id],
onDelete: Cascade)

  @@index([accountId, batchId, occurredAt])
  @@index([batchId, occurredAt])
}

model Subscription {
  id                   String             @id @default(uuid())
  userId               String
  provider             String
  status               SubscriptionStatus
  planCode             String
  stripeCustomerId     String?
  stripeSubscriptionId String?
  currentPeriodEnd     DateTime?
  createdAt            DateTime           @default(now())
  updatedAt            DateTime           @updatedAt

  user                 User               @relation(fields: [userId],
references: [id], onDelete: Cascade)
```

```
  @@index([userId])
  @@index([provider, stripeSubscriptionId])
}

model Entitlement {
  id         String   @id @default(uuid())
  userId     String   @unique
  planCode   String
  features   Json
  limits     Json
  validUntil DateTime?
  updatedAt  DateTime @updatedAt
  createdAt  DateTime @default(now())

  user       User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
}

model StripeEvent {
  id          String   @id
  type        String
  livemode    Boolean
  receivedAt  DateTime @default(now())
  processedAt DateTime?
  payload     Json

  @@index([type])
}
```

Appendix B - Local development Docker Compose (outline)

```
services:
  nginx:
    image: nginx:stable
    ports:
      - "8080:80"
```

```yaml
    volumes:
      - ./infra/nginx/dev.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on:
      - web
      - api

  web:
    build: ./apps/web
    command: npm run dev
    environment:
      - NEXT_PUBLIC_API_BASE_URL=/api
    ports:
      - "3000:3000"

  api:
    build: ./services/api
    command: npm run dev
    environment:
      - DATABASE_URL=postgresql://postgres:postgres@postgres:5432/brewapp
      - STRIPE_WEBHOOK_SECRET=...
    ports:
      - "4000:4000"
    depends_on:
      - postgres

  postgres:
    image: postgres:16
    environment:
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_DB=brewapp
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Appendix C - CI outline (GitHub Actions)

Minimum pipeline stages:

1. install + cache
2. typecheck (TS)
3. unit tests (Vitest)
4. build (web + api)
5. e2e (Playwright) against a compose stack (optional early, recommended soon)